

Battery SOH & RUL Prediction Code Overview

Introduction

This document provides a detailed overview of the `battery_simple.py` script, explaining its structure, functions, and how the code executes the battery health prediction pipeline. The script combines data generation, machine learning model training, evaluation, visualization, and a user interface into a single executable.

Code Structure

The script is organized into several logical sections:

1. Imports and setup
2. Constants and global state
3. Data generation and preprocessing functions
4. Model building and training functions
5. Evaluation and plotting functions
6. User interface class
7. Main execution

Imports and Setup

```
import os, numpy as np, matplotlib.pyplot as plt, warnings
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
warnings.filterwarnings('ignore')
import tensorflow as tf
tf.get_logger().setLevel('ERROR')
from tensorflow import keras
from tensorflow.keras import layers, Model
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox
import threading
from pathlib import Path
np.random.seed(42); tf.random.set_seed(42)
```

- TensorFlow and Keras are used for deep learning.
- Scikit-learn provides preprocessing and metrics.
- Tkinter creates the GUI.
- Random seeds ensure reproducible results.

Constants and Global State

```
NOMINAL = 1.1 # nominal capacity in Ah
EOL = 0.88 # end-of-life threshold (80% of nominal)
WINDOW = 10 # use last 10 cycles to predict next SOH
scaler_X = MinMaxScaler()
scaler_Y = MinMaxScaler(feature_range=(0.05, 0.95))
DATA = {}
results = {}
history = None
PLOT_FILES = {
    'Degradation': 'plot1_degradation.png',
    'SOH prediction': 'plot2_soh.png',
    'RUL prediction': 'plot3_rul.png',
    'Comparison': 'plot4_comparison.png',
    'Pre-training loss': 'plot5_loss.png',
}
```

- `NOMINAL` and `EOL` define battery health thresholds.
- `WINDOW` sets the sequence length for LSTM input.
- Scalers normalize features and targets.
- Global dictionaries store data and results.
- `PLOT\_FILES` maps plot names to filenames.

Data Generation Functions

```
### generate_battery(n_cycles, seed=0)
```

Generates synthetic capacity data for a battery:

- Uses an exponential decay model: $\text{cap} = 1.1 * \text{np.exp}(-0.0004 * c - 1.5e-7 * c ** 2)$
- Adds recovery humps at 25% and 52% of cycles.
- Includes Gaussian noise for realism.
- Clips capacity between 0.82 and 1.1 Ah.

```
### extract_features(capacity)
```

Creates 5 features per cycle from capacity data:

1. Voltage mean: simulated from SOH-dependent voltage curve.
2. Voltage std: standard deviation of simulated voltage.

3. Current mean: constant current with noise.
4. CC charge time: proportional to SOH.
5. CV charge time: proportional to SOH.

Features are synthetic but representative of real battery measurements.

```
### initialize_data(log=print)
```

- Calls `generate\_battery` for four batteries (CS35–CS38).
- Computes SOH and EoL for each.
- Extracts features and stores in global `DATA` dict.
- Fits scalars on source battery data.

Sequence Preparation

```
### make_sequences(features_norm, soh_norm, window=WINDOW)
```

- Creates sliding window sequences for LSTM input.
- X: shape `(samples, window, n\_features)`
- y: next SOH value to predict.

```
### prepare_battery(name, train_split)
```

- Normalizes features and SOH for a battery.
- Splits data into training and test sequences.
- Returns training data, test data, true SOH, and training size.

Model Building and Training

```
### build_lstm()
```

Constructs the LSTM model:

- Input: `(WINDOW, 5)` sequences.
- LSTM(64) with return_sequences=True.
- Dropout(0.2).
- LSTM(32).
- Dropout(0.2).
- Dense(1) output.

```
### fine_tune_predict(split, label, log=print)
```

Performs transfer learning fine-tuning:

1. Loads pre-trained weights.
2. Freezes first LSTM layer.

3. Trains upper layers with Adam(0.001) for 60 epochs.
4. Unfreezes all layers.
5. Fine-tunes with Adam(0.0005) for 25 epochs.
6. Predicts SOH and computes metrics (MAE, RMSE, MAPE).

run_experiment(log=print)

Orchestrates the full pipeline:

1. Initializes data.
2. Pre-trains on source batteries (Phase A).
3. Fine-tunes on CS38 at 10%, 50%, 80% splits (Phase B).
4. Trains scratch baseline on 10% CS38.
5. Computes RUL from SOH predictions.
6. Generates all plots.

Evaluation and Plotting

soh_to_rul(soh_array, threshold=0.80)

Converts SOH to RUL:

- Finds EoL cycle where SOH < threshold.
- Computes remaining cycles for each point.

Plot Generation

Five plots are created using Matplotlib:

1. **Degradation**: Capacity curves for all batteries.
2. **SOH prediction**: True vs predicted SOH at different splits.
3. **RUL prediction**: True vs predicted RUL with error shading.
4. **Comparison**: MAE bar chart for all models.
5. **Pre-training loss**: Training and validation loss curves.

Plots are saved as PNG files and can be opened from the UI.

User Interface

BatteryUI Class

A Tkinter application with:

- Control frame: Run, Clear Log, and plot buttons.
- Status label: Shows current operation.
- Scrolled text: Displays log messages.
- Threading: Runs experiment in background to keep UI responsive.

Key Methods

- `log(message)`: Updates status and log text.
- `start_run()`: Launches experiment thread.
- `run_pipeline()`: Executes `run_experiment` with error handling.
- `open_plot(filename)`: Opens plot files using `os.startfile`.

Execution Flow

When python battery_simple.py is run:

1. `main()` creates and starts the Tkinter app.
2. User clicks "Run Experiment".
3. `run_experiment` is called in a thread.
4. Data is generated and processed.
5. Model is pre-trained and fine-tuned.
6. Plots are saved.
7. Results are logged.
8. User can open plots via buttons.

Dependencies

- Python 3.11+
- TensorFlow/Keras
- NumPy, Matplotlib, Scikit-learn
- Tkinter (built-in)
- PIL (for screenshot in report generation)

Key Design Decisions

- **Synthetic data**: Allows reproducible testing without real datasets.
- **Transfer learning**: Demonstrates domain adaptation for battery prediction.
- **Sliding windows**: Captures temporal dependencies in degradation.
- **Early stopping**: Prevents overfitting during training.
- **GUI**: Makes the script accessible without command-line knowledge.
- **Threading**: Keeps UI responsive during long training runs.

Potential Improvements

- Add real data loading from CSV files.
- Implement model saving/loading for different batteries.
- Add hyperparameter tuning options in UI.

- Include more evaluation metrics and cross-validation.
- Support for different model architectures.

Code overview generated on April 23, 2026.